Sistemas Basados en Microprocesador

Bloque 2 – Práctica 1

Departamento de Ingeniería Telemática y Electrónica
Universidad Politécnica de Madrid

Sistema operativo en tiempo real. Introducción a CMSIS-RTOS V2

---

1. Introducción y objetivos de la práctica

El objetivo de esta práctica es, en una primera parte, familiarizarse con el uso de una de las funcionalidades básicas del sistema operativo CMSIS-RTOS V2 (RTX5), como es la gestión de threads y la sincronización entre ellos, en un microcontrolador Cortex-M. El uso de un RTOS representa un enfoque de diseño de aplicaciones más sofisticado, que fomenta el desarrollo de código estructurado mediante el uso de su API y ayuda a obtener un comportamiento más predecible del sistema. La utilización de un sistema operativo en tiempo real proporciona soporte multithread y mayores garantías en la consecución de latencias acotadas en aplicaciones de tiempo real, incluso en pequeños microcontroladores como el que se está utilizando. En una segunda parte, haciendo uso de los conocimientos adquiridos, se desarrollará un driver software para manejar el joystick disponible en la tarjeta mbed application board, apoyándose en los recursos básicos que ofrece el sistema operativo desde la API CMSIS-RTOS2: threads, timers virtuales, señalización entre threads, etc. Se manejarán los cinco gestos disponibles en el joystick de la mbed application board y, se busca disponer de una serie de funciones de librería en ficheros .c y .h para poder utilizarlas en futuros proyectos y en el diseño final. Por tanto, es muy importante depurar y obtener un "driver"libre de errores que permita al usuario interactuar correctamente con el microcontrolador. Antes de comenzar a desarrollar la práctica es de suma importancia que lea la documentación proporcionada en clase así como los ejemplos disponibles en el repositorio SBM-rtos.

2. Material

Además de los documentos disponibles en Moodle debe de utilizar:

- Tarjeta de desarrollo NUCLEO STM32F429Zi
- Cable USB a micro-USB
- CMSIS-RTOS2 Documentation

3. Git

El código debe almacenarse en la carpeta indicada en la 1. Cada paso de ejercicio debe tener el tag indicado.

Carpeta	B2\P2_1
Tag	B2_P1_Ej1
Tag	B2_P1_Ej2
Tag	B2_P1_Ej3
Carpeta	B2\P2_2
Tag	B2_P1_Ej4
Tag	B2_P1_Ej5

Tabla 1: Carpeta y tags a utilizar en la práctica

4. Ejercicios

Ejercicio 1

Cree un proyecto nuevo (P1_1 del B2) para la tarjeta de desarrollo, utilizando CMSIS-RTOS. Añada dos ficheros fuente, main.c y Thread.c, haciendo uso de los "User Code Template". Analice el contenido detallado de main.c y compárelo con el creado en prácticas anteriores sin el uso del sistema operativo. ¿Qué diferencias encuentra en el código? Cree el código necesario para tener un único thread (denominado Thled1) en la aplicación. Dicho thread debe encargarse de encender y apagar periódicamente el LED1 de la tarjeta, de forma que este permanezca encendido 200 ms y apagado 800 ms en cada ciclo. Haga uso de la función osDelay. Utilice el analizador lógico para comprobar que los tiempos han sido generados correctamente. Utilizando el "wizard" de configuración del fichero RTX_Config.h `cambie el valor del tick del sistema para que los tiempos antes indicados sean la mitad`. El código fuente no debe ser modificado. ¿Qué parámetros ha modificado?

Una vez comprobado el correcto funcionamiento del ejercicio, añada a git los ficheros correspondientes, haga el commit y el push y haga el tag siguiendo el criterio definido al comienzo de este documento.

Ejercicio 2

Añada dos nuevos ficheros fuente denominados Thled2.c y Thled3.c. Cada fichero fuente se encargará de manejar un nuevo thread denominados Thled2 y Thled3 respectivamente. Asegúrese que el tick del sistema operativo vuelve a ser de 1 ms. El thread Thled2 debe encargarse de encender y apagar periódicamente el LED2 de la tarjeta, de forma que este permanezca encendido 137 ms y apagado 137 ms. El thread Thled3 debe encargarse de encender y apagar periódicamente el LED3 de la tarjeta, de forma que este permanezca encendido 287 ms y apagado 287 ms. Todos los threads deben estar activos desde el inicio de la aplicación de manera que los tres se ejecuten de manera concurrente. Utilice el analizador lógico para comprobar que los tiempos han sido generados correctamente. En el thread main, una vez creados los tres threads coloque un punto de ruptura y utilice la herramienta de visualización del sistema operativo (View->Watch Windows->RTX RTOS) para analizar el comportamiento de la aplicación. Rellene la tabla 2

Número de Threads de la aplicación:		
Nombre del Thread	**Estado del mismo**	**Prioridad del mismo**
ThLed1	osThreadReady	osPriorityNormal
ThLed2	osThreadReady	osPriorityNormal
ThLed3	osThreadReady	osPriorityNormal

Tabla 2: Threads de la aplicación

Cambie el punto de ruptura y sitúelo en el comienzo del thread Thled2. Rellene la tabla 3.

Nombre del Thread	Estado del mismo	Prioridad del mismo
ThLed1	osThreadBlocked	osPriorityNormal
ThLed2	osThreadRunning	osPriorityNormal
ThLed3	osThreadReady	osPriorityNormal

Tabla 3: Threads de la aplicación

Pruebe a cambiar el punto de ruptura entre los diferentes threads y verifique cómo evoluciona el estado de los threads mediante la herramienta de análisis. ¿Se le ocurre alguna manera de simplificar la implementación de esta práctica a partir del código disponible en los ejemplos localizados en `https://github.com/Univ-Politecnica-Madrid/SBM`? No es necesario que lo implemente.

Una vez comprobado el correcto funcionamiento del ejercicio, añada a git los ficheros correspondientes, haga el commit y el push y haga el tag siguiendo el criterio definido al comienzo de este documento.

Ejercicio 3

En este último ejemplo de esta práctica se pretende sincronizar la ejecución de los tres threads definidos anteriormente mediante el uso de flags (Thread Flags de CMSIS-RTOS2). La secuencia en la que deben ejecutarse los threads es la definida a continuación:

- Cuando la aplicación arranca, los LED 2 y 3 permanecen apagados inicialmente. El LED 1 comenzará a parpadear según lo indicado en el Ejemplo 1 y se mantendrá así indefinidamente.
- Después de los primeros cinco ciclos del LED1, el LED2 deberá comenzar su parpadeo durante 20 ciclos, con la cadencia indicada en el Ejemplo 2. Pasados esos 20 ciclos se detendrá.
- Después de 15 ciclos del LED2 el LED3 deberá comenzar su parpadeo durante 30 ciclos, con la cadencia indicada en el Ejemplo 2. Pasados esos 30 ciclos se detendrá hasta una nueva activación tras 15 nuevos ciclos del LED2.
- Después de 25 ciclos del LED3 el LED2 deberá comenzar nuevamente su parpadeo durante otros 20 ciclos.
- Los puntos 3 y 4 se repetirán indefinidamente.

La figura 1 representa esquemáticamente la secuencia temporal que indica los instantes en que cada uno de los tres LEDs deben estar parpadeando.

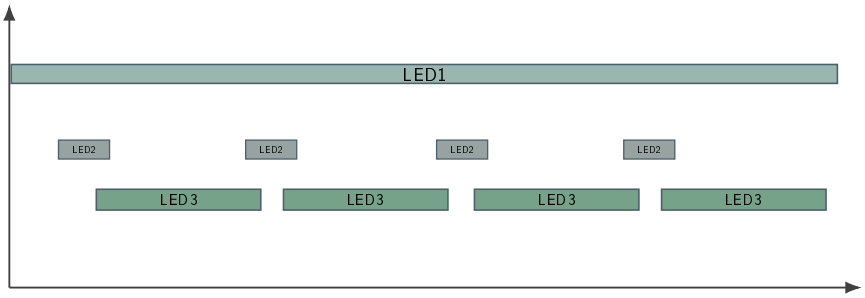


Figura 1: Evaluación temporal de la activación de los LEDs

Ejecute el código y verifique su funcionamiento. Haga uso de la herramienta "View->Watch Windows->RTX RTOS" para verificar el estado en que se encuentran los threads de su aplicación.

> Una vez comprobado el correcto funcionamiento del ejercicio, añada a git los ficheros correspondientes, haga el commit y el push y haga el tag siguiendo el criterio definido al comienzo de este documento.

Ejercicio 4

Utilizando la tarjeta NUCLEO-144 STM32F429ZI desarrolle un proyecto (P1_2 del B2) que responda a los siguientes requisitos:

- Se utilizará el botón AZUL para generar interrupciones externas.
- Se utilizarán los LEDs rojo (LD3) y verde (LD1) para mostrar el estado del sistema.
- En el arranque del sistema se encenderá el led verde y se arrancará una cuenta de tres segundos. Pasados esos tres segundos sin que se pulse nuevamente el gesto RIGHT el sistema apagará el led verde y encenderá el led rojo.
- Si se pulsa el botón AZUL mientras la cuenta de tres segundos está activa, se reiniciará un nuevo periodo de cuenta de tres segundos.
- Una vez que el sistema se bloquea (LED rojo encendido), no se procesarán peticiones de reinicio de la cuenta con el botón AZUL, solo un reset del microcontrolador hace que el sistema vuelva a la situación inicial.

En este ejercicio la temporización debe ser obtenida mediante **timers virtuales**. Tenga en cuenta que las funciones de manejo de timers no se pueden utilizar desde rutinas de atención a la interrupción.

> Una vez comprobado el correcto funcionamiento del ejercicio, añada a git los ficheros correspondientes, haga el commit y el push y haga el tag siguiendo el criterio definido al comienzo de este documento.

Ejercicio 5

El joystick incluye cinco pulsadores encapsulados en un mismo dispositivo. Es conveniente recordar que los pulsadores presentan el conocido efecto de los rebotes, debido al choque de las láminas metálicas con los que están construidos. En las siguientes figuras se muestran dos ejemplos de este fenómeno.

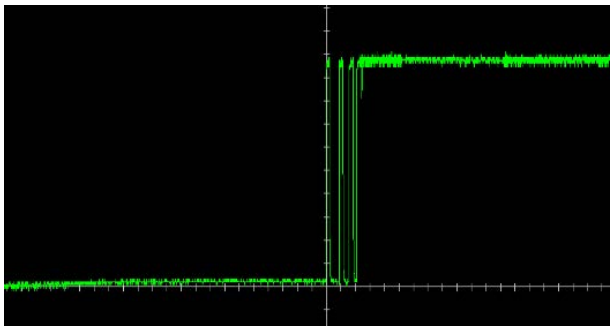


Figura 2: Enter Caption

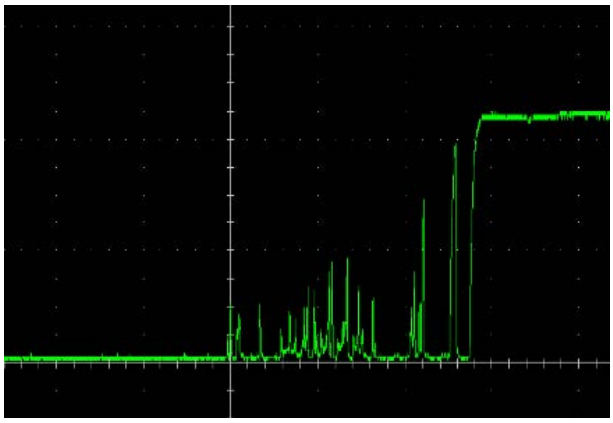


Figura 3: Enter Caption

A la vista de las figuras se puede observar cómo transcurre un tiempo hasta que el valor de la pulsación es estable. El número de rebotes y el tiempo que transcurre entre ellos dependen del tipo y estado del pulsador. Asimismo, una persona necesita un tiempo mínimo para actuar sobre un pulsador. Teniendo estas consideraciones presentes, se va a seguir la siguiente estrategia para gestionar el joystick.

- Cada vez que se acciona cualquier pulsador (o gesto) del joystick, esto debe generar en nuestro sistema una interrupción.
- Una vez detectada la interrupción se enviará una señal a un thread que estará esperando la llegada de una señal procedente de la callback del manejador de interrupciones del joystick. Una vez recibida la señal se arrancará un timer one-shot con una duración de 50 ms. De esta manera, cada vez que se produzca un flanco (rebote) en el joystick , se generará una nueva interrupción y el timer se reiniciará de nuevo. Una vez que venza la temporización se puede dar por finalizado el proceso de pulsación y se puede leer el estado de las cinco líneas del joystick y determinar qué gesto de este se ha accionado.
- Una vez que se haya leído el joystick, incremente una variable por cada gesto pulsado, de forma que se pueda llevar la cuenta del número de gestos accionados en el joystick. Asimismo, represente en los leds de la tarjeta la tecla pulsada codificada de la siguiente forma.

Gesto	Valor
Arriba	1
Derecha	2
Abajo	3
Izquierda	4
Centro	5

Tabla 4: Valores asignados a cada gesto

Para la representación del código del gesto seleccionado en los LEDs debe considerar que LD1 es el LSB. Conecte el joystick de la tarjeta de aplicaciones (mbed app board) a la tarjeta NUCLEO-144 STM32F429ZI siguiendo la siguiente tabla:

Gesto *joystick*	**Puerto/Pin tarjeta Núcleo**	**Pin *mbed app board***
Arriba	Puerto B / Pin 10	15
Derecha	Puerto B / Pin 11	16
Abajo	Puerto E / Pin 12	12
Izquierda	Puerto E / Pin 14	13
Center	Puerto E / Pin 15	14

Tabla 5: Asignación de pines según gesto del joystick

Implemente un driver para la gestión del joystick, El driver estará formado por un módulo software que se denominará Thjoy.c que implementará la estrategia antes indicada. El módulo estará formado por un thread, un timer virtual, así como las señales necesarias para la correcta sincronización.

Una vez comprobado el correcto funcionamiento del ejercicio, añada a git los ficheros correspondientes, haga el commit y el push y haga el tag siguiendo el criterio definido al comienzo de este documento.